

day 16

node day 2

أهلاً بك يا حنين! 🚀

جهّزت لك ملخص اليوم الخاص بـ **File System & HTTP Server (Lab 2)** بتنسيق **Obsidian Markdown** منظم وشامل لكافة مفاهيم السيشن المرفقة، مع الكود الكامل الجاهز للرفع على **GitHub** والتسليم لـ **Part 1** و **Part 2** متضمناً كافة النقاط الـ Bonus!

📌 01. Obsidian Notes: Node.js Core Modules & Event Loop

💡 المفهوم الأساسي تعتمد بيئة **Node.js** على **V8 Engine** مع مكتبة **libuv** لمعالجة العمليات اللا متزامنة (Asynchronous Non-blocking I/O) وإدارة الـ **Event Loop** والموديلز الأساسية المدمجة مثل **fs** و **http**.

1. Arrow Functions vs Normal Functions (**this** binding)

JavaScript

```
const object = {
  name: "mo",
  statementFunction: function () {
    console.log(this); // 🟢 { name: 'mo', ... }
  },
  arrowFunction: () => {
    console.log(this); // 🟡 Outer Lexical Scope (Global/Module Scope)
  },
};
```

2. Modern JavaScript Essentials

♦ Array Methods Quick Reference:

- **map**: جديدة بتطبيق دالة على كل عنصر **Array** ينشئ.
- **filter**: يرجع العناصر التي تحقق شرطاً معيناً.
- **reduce**: (Object رقم، نص، أو) يختزل العناصر في قيمة واحدة.
- **some / every**: يرجع **true / false** على تحقق الشرط لبعض أو كل العناصر.
- **findIndex**: الخاص بأول عنصر يطابق الشرط (أو -1) **index** يرجع الـ.

◆ Rest & Spread Operators:

- **Spread** (`...`): يفرد عناصر Array أو Object.
- **Rest** (`...`): واحدة Array يجمع المعلومات الممررة للدالة داخل.

● 3. Event Loop Execution Order (Execution Sequence)

ترتيب تنفيذ الكود الا متزامن في Node.js يتم كالتالي:

1. **Synchronous Code** (ينفذ فوراً).
2. **process.nextTick()** (Microtasks أعلى أولوية في الـ).
3. **Promises** (`Promise.then`) (مباشرة nextTick تلي الـ).
4. **Timers Phase** (`setTimeout`).
5. **Check Phase** (`setImmediate`).
6. **I/O Callbacks** (`fs.readFile`).

JavaScript

```
console.log("start");
setTimeout(() => console.log("set timeout"), 0);
setImmediate(() => console.log("set immediate"));
process.nextTick(() => console.log("process next tick"));
Promise.resolve().then(() => console.log("promise resolve"));

// 🚦 Output Order:
// 1. start
// 2. process next tick
// 3. promise resolve
// 4. set timeout
// 5. set immediate
```

● 4. File System Module (`fs`) & JSON Methods

- `JSON.parse(jsonString)` : تحويل نص إلى JavaScript Object/Array.
- `JSON.stringify(jsObject)` : تحويل JavaScript Object إلى نص JSON.
- **Sync vs Async File Operations:**
 - **Synchronous** (`readFileSync`): يُعطل تنفيذ الكود (Blocking) حتى انتهاء القراءة.
 - **Asynchronous** (`readFile`): (Non-blocking) لا يُعطل الكود، ويعتمد على الكولباك أو الـ Promises.

أسئلة إنترفيو متوقعة (Interview Q&A) ?

② **Q1: Why do we parse request body chunk by chunk in native**

Stream of Data يأتي على هيئة **Request Body** الإجابة: لأن الـ `http.createServer` لتجميع الأجزاء، وعند انتهاء الاستلام في `req.on('data')` عبر الشبكة. نقوم بالاستماع لأحداث **Chunks** `JSON.parse()` باستخدام **Object** نقوم بتحويل السلسلة الناتجة إلى `req.on('end')`.

② **Q2: What is the benefit of `path.join(__dirname, "students.json")` ?** الإجابة:

(`Linux/Mac` / `Windows` \ `Linux/Mac`) تضمن تعيين المسار المطلق بشكل صحيح متوافق مع كافة أنظمة التشغيل لتجنب أخطاء عدم العثور على الملف عند التشغيل من مسارات مختلفة.